

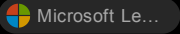
I'll map your AWS Bedrock architecture to the closest Microsoft Foundry/Azure equivalents, then lay out an implementation order you can follow without waiting on AWS support. I'll ground the steps in current Microsoft documentation and RAG architecture guidance.

Thought for 16s >

Yes — you can build a close Microsoft Foundry equivalent of your Amazon Bedrock RAG architecture, and it is a good idea while you wait on AWS support.

1. AWS-to-Microsoft Foundry architecture mapping

Your AWS Bedrock component	Microsoft Foundry / Azure equivalent	Purpose
S3 bucket	Azure Blob Storage container	Stores your source policy documents: PDFs, Word files, Markdown, text, etc.
S3 vector store / S3 Vectors	Azure AI Search vector index	Stores searchable chunks, metadata, and embedding vectors.
Titan Text Embeddings V2	Azure OpenAI / Foundry embedding model, usually <code>text-embedding-3-small</code> or <code>text-embedding-3-large</code>	Converts policy chunks into vectors.
Bedrock Knowledge Base	Azure AI Search index + Microsoft Foundry knowledge base / Foundry IQ / Agent file search	Retrieval layer used to ground chat answers in your documents.
Bedrock Agent / KB test chat	Microsoft Foundry Agent or Chat Playground	User-facing policy assistant experience.

Microsoft describes RAG in Foundry as a pattern where the application retrieves relevant data from an index, augments the prompt with that grounding data, and then generates an answer with citations. 

Recommended implementation order

For your **Policy AI Assistant**, I recommend this order:

1. Create Azure resource group
2. Create Azure Blob Storage container for policy documents

3. Create Microsoft Foundry project
4. Deploy embedding model
5. Deploy chat/completion model
6. Create Azure AI Search service
7. Create vector index using integrated vectorization
8. Ingest policy documents from Blob Storage
9. Create Foundry knowledge base / attach index to agent
10. Test retrieval quality in Chat Playground or Agent Playground
11. Tune chunking, search mode, and prompts

This gives you the closest equivalent to:

S3 bucket → Titan embeddings → Bedrock Knowledge Base → RAG assistant

Target architecture for your Azure RAG Policy Assistant



```
graph TD; PD[Policy Documents] --> ABS[Azure Blob Storage Container]; ABS --> AISI[Azure AI Search Indexer]; AISI --> CES[Chunking + Embedding Skill]; CES --> AIVS[Azure AI Search Vector Index]; AIVS --> MFKB[Microsoft Foundry Knowledge Base / Agent File Search]; MFKB --> PAACE[Policy AI Assistant Chat Experience];
```

Azure AI Search supports RAG workloads with both **classic RAG** and newer **agentic retrieval**. Microsoft recommends agentic retrieval for new RAG implementations when the client is an agent or chatbot and you want better relevance, citations, and structured grounding data.  Microsoft Le...

For your first version, I would start with **classic Azure AI Search RAG using Blob Storage + vector index**, then move to **Foundry IQ / agentic retrieval** once the basic pipeline works.

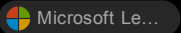


Phase 1 — Confirm quota and region first

Before building anything, check whether your Azure subscription has usable quota for the models you need.

You should confirm quota for:

Model type	Suggested model	Why
Embeddings	<code>text-embedding-3-small</code>	Lower cost, good enough for most policy RAG demos
Optional higher-quality embeddings	<code>text-embedding-3-large</code>	Better retrieval quality, higher vector dimensions/cost
Chat model	<code>gpt-4o-mini</code> , <code>gpt-4.1-mini</code> , or similar	Good for policy assistant demos

Azure OpenAI / Foundry quota is controlled by **tokens per minute** and **requests per minute**, and Microsoft says these limits are scoped by **region, subscription, model, and deployment type**. 

What to do

In **Azure AI Foundry**:

1. Open your Foundry project.
2. Go to **Management**.
3. Go to **Quota**.
4. Check available quota for:
 - Embedding models
 - Chat models
 - Your selected region

Because you are using an Azure trial/free subscription, this step is important. Free or trial subscriptions can have limited or unavailable quota for some Azure OpenAI models.



Phase 2 — Create the policy datastore

Azure equivalent of S3 bucket: Azure Blob Storage

Create:

- Resource group: `rg-policy-ai-rag-dev`

- Storage account: `stpolicyairagdev<unique>`
- Blob container: `policy-documents`

Recommended initial settings:

Setting	Recommendation
Redundancy	Locally redundant storage, LRS, for demo/dev
Public access	Disabled
Container access	Private
Hierarchical namespace	Optional; not required for first RAG demo
Versioning	Optional, but useful later
Soft delete	Recommended
Encryption	Microsoft-managed keys are fine for demo

Upload a small test set first:

```
remote-work-policy.pdf
incident-management-process.pdf
password-reset-guide.pdf
cloud-cost-policy.pdf
employee-onboarding-policy.pdf
```



Start small because your earlier AWS issue was likely caused by ingestion-time embedding throughput. A small Azure test set lets you verify the architecture before scaling.

Azure AI Search indexers can import content from Azure Blob Storage into a search index. Microsoft's Blob indexer documentation says the indexer takes blobs from one container as input and outputs searchable content and metadata into a search index.  Microsoft Le...



Phase 3 — Create the Microsoft Foundry project

In Azure AI Foundry:

1. Go to **Azure AI Foundry**.
2. Create a new project.
3. Name it something like:



4. Use the same region as your models and search service when possible.
5. Connect or create the required Azure AI services resource.

Microsoft's RAG tutorial for Foundry has you create a project, create an Azure AI Search index, install Azure CLI/Python, deploy models, and configure environment variables.



For your first build, you can do most of this through the portal instead of code.



Phase 4 — Deploy the embeddings model

Recommended embeddings model

Use:

`text-embedding-3-small`

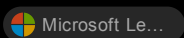


Why:

- Good for RAG demos.
- Lower cost than `text-embedding-3-large`.
- Good enough for policy documents.
- Avoids overbuilding the first version.

Use `text-embedding-3-large` later only if retrieval quality is weak.

Microsoft's embeddings tutorial lists `text-embedding-3-small`, `text-embedding-3-large`, and `text-embedding-ada-002` as embedding models used for embeddings workflows.



Important dimension note

In AWS, you were deciding between Titan V2 dimensions such as 1024 or 512.

In Azure, the same rule applies:

The vector index dimension must match the embedding model output dimension.

Azure AI Search vector indexes require vector fields to be configured correctly, and the vector field is part of the index schema.



For `text-embedding-3-small`, use the dimension value supported by your Foundry deployment/index wizard. If the portal auto-generates the index, let the wizard set this for

you.



Phase 5 — Deploy the chat model

For your first Policy AI Assistant, deploy one of these:

Model	Use case
<code>gpt-4o-mini</code>	Good low-cost assistant model
<code>gpt-4.1-mini</code>	Stronger reasoning, still efficient
<code>gpt-4.1</code> or newer larger model	Better quality, higher cost/quota need

For your demo project, I would start with:

`gpt-4o-mini` or `gpt-4.1-mini`



Do not start with the largest model. Your immediate goal is proving the RAG architecture, not maximizing model quality.



Phase 6 — Create Azure AI Search service

Azure equivalent of S3 vector store

Create an **Azure AI Search** service.

Recommended first settings:

Setting	Recommendation
Pricing tier	Basic or Standard, depending on availability
Region	Same region as Foundry/OpenAI if possible
Semantic ranker	Enable if available
Public network access	OK for demo; restrict later
Authentication	Key-based for first demo; managed identity later

Azure AI Search is Microsoft's main managed vector-search service for RAG. Microsoft says Azure AI Search supports indexing, storing, and querying vector embeddings from a

For your first implementation, Azure AI Search is the best match to your AWS S3 Vectors / Bedrock Knowledge Base vector storage layer.

Phase 7 — Create the vector index and ingestion pipeline

You have two implementation choices.

Option A — Portal-first approach, recommended for you now

Use the Azure AI Search or Foundry “import and vectorize” workflow.

High-level steps:

1. Open **Azure AI Search**.
2. Select your search service.
3. Choose **Import and vectorize data**.
4. Select **Azure Blob Storage** as the data source.
5. Select your `policy-documents` container.
6. Choose your embedding model deployment.
7. Let the wizard create:
 - Data source
 - Skillset
 - Index
 - Indexer
 - Vector field
 - Chunking configuration
8. Run the indexer.

Microsoft’s integrated vectorization documentation says Azure AI Search can use a skillset to chunk and vectorize content from a supported data source, call a text-splitting or document-layout skill, call an embedding skill, and store the result in a vector index.

This is the closest Azure equivalent to Bedrock Knowledge Base ingestion.

Option B — Code-first approach, later

Use REST, Python, or Azure SDKs to create:

- Data source
- Index schema
- Skillset
- Indexer
- Vectorizer
- Semantic configuration
- Query pipeline

This gives you more control, but I would not start here unless the portal fails.

Phase 8 — Recommended chunking strategy

For your first Azure build, use conservative chunking.

Because you had AWS 429 throttling during ingestion, do **not** start with aggressive small chunks.

Recommended initial policy-document chunking:

Setting	Recommendation
Chunk size	800–1,200 tokens
Overlap	10–15%
Start document count	5–10 files
Search type	Hybrid search if available
Semantic ranker	Enable if available
Metadata fields	document name, policy category, effective date, source path

Microsoft's Foundry vector-store file-search defaults are 800-token chunks, 400-token overlap, `text-embedding-3-large` at 256 dimensions, and up to 20 chunks added to context. 

For your use case, I would reduce overlap from that default if the portal allows it, because policy documents can produce a lot of redundant chunks. Start with:

Chunk size: 1000 tokens
Overlap: 100–150 tokens



That should reduce ingestion load compared with very small chunks and large overlap.

Phase 9 — Create the Microsoft Foundry knowledge base / retrieval layer

There are two valid paths here.

Path 1 — Classic RAG with Azure AI Search index

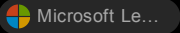
Use this first.

Flow:

```
User question
  → Azure AI Search hybrid/vector search
  → Top policy chunks
  → Chat model prompt
  → Grounded answer with citations
```



This is simpler and more stable.

Azure AI Search documentation says classic RAG uses the original search pipeline where your app sends a query to Azure AI Search and then gives the result set to the LLM for answer generation. 

Path 2 — Foundry IQ / agentic knowledge base

Use this after the basic index works.

Microsoft says an Azure AI Search **knowledge base** is a top-level object for agentic retrieval. It defines which knowledge sources to query and the default retrieval behavior.



This is closer to the newer “knowledge base” concept in Foundry.

For your policy assistant, the agentic version would look like this:

```
Policy documents in Blob Storage
  → Azure AI Search index / knowledge source
  → Foundry knowledge base
  → Foundry Agent
  → Policy AI Assistant
```



I would not start with this until your Azure AI Search index is working.

Phase 10 — Attach your index to Chat Playground or Agent Playground

In Microsoft Foundry:

1. Go to your project.
2. Open **Chat Playground** or **Agents**.
3. Select your deployed chat model.
4. Add your data source / index.
5. Choose your Azure AI Search index.
6. Configure retrieval mode:
 - Start with **hybrid search** if available.
 - Enable semantic ranking if available.
7. Test with policy questions.

Sample test questions:

What is the company policy for remote work?

What should an employee do if they need a password reset?

What is the incident escalation process?

What are the rules for cloud cost approvals?

Which document did you use to answer this question?



Your assistant should cite the source policy document. If it gives an answer without a source, tighten the system prompt.

Recommended system prompt for your Policy AI Assistant

Use something like this:

You are a Policy AI Assistant. Answer questions only using the retrieved policy documents. If the answer is not found in the retrieved documents, say that the policy source could not be found. Do not guess. Include citations or source document names whenever possible. Use clear, professional language suitable for employees and managers.



This is important because RAG quality depends not only on the vector database, but also on the grounding prompt. Microsoft's Foundry RAG documentation identifies grounding

data, indexes, embeddings, system messages, and prompts as key RAG concepts.



Practical first-build checklist

Use this as your implementation checklist.



Step 1 — Azure setup

Create:

```
Resource group: rg-policy-ai-rag-dev
Region: same region where you have model quota
Storage account: stpolicyairagdev<unique>
Blob container: policy-documents
Azure AI Search service: srch-policy-ai-rag-dev
Foundry project: policy-ai-assistant-dev
```



Step 2 — Upload test documents

Upload 5–10 documents only.

Suggested folder structure:

```
/policies/hr/
/policies/it/
/policies/security/
/policies/cloud/
/policies/operations/
```



Step 3 — Deploy models

Deploy:

```
Embedding model: text-embedding-3-small
Chat model: gpt-4o-mini or gpt-4.1-mini
```



Step 4 — Create vector index

Use Import and vectorize data.

Create index:

```
policy-rag-index
```



Recommended fields:

```
id
content
content_vector
source_document
source_path
policy_category
last_modified
page_number
chunk_id
```



Step 5 — Run indexer

Run the indexer and confirm:

- Documents were discovered.
- Chunks were created.
- Embeddings were generated.
- Vector index contains records.
- No quota or 429 errors occurred.

Step 6 — Test retrieval

Ask specific questions and verify:

- Correct document retrieved.
- Correct chunk retrieved.
- Answer cites the source.
- Assistant says “I don’t know” when policy is missing.

Step 7 — Scale slowly

After 5–10 files work, increase to:

```
25 files → 50 files → 100 files
```



Do not upload the full policy library all at once until you know your Azure embedding quota can handle ingestion.



My recommendation for your first Azure build

For your situation, I recommend this exact target:

```
Datastore:
Azure Blob Storage

Vector database:
```



Azure AI Search vector index

Embedding model:

text-embedding-3-small

Chat model:

gpt-4o-mini or gpt-4.1-mini

Retrieval mode:

Hybrid search + semantic ranking if available

Knowledge layer:

Start with Azure AI Search index connected to Foundry Chat Playground.

Then move to Foundry Agent / Foundry IQ knowledge base after the basic RAG pipeline

This gives you the Microsoft equivalent of your AWS architecture, but with more control and better visibility into each layer.

Key difference from AWS Bedrock Knowledge Bases

The biggest difference is this:

Amazon Bedrock Knowledge Base hides more of the ingestion pipeline. Microsoft Foundry + Azure AI Search exposes more of it.

That means you will have more moving parts:

```
Blob Storage
Azure AI Search data source
Indexer
Skillset
Embedding deployment
Vector index
Foundry project
Chat/Agent layer
```



But you also get more control over:

- Chunking
- Index schema
- Metadata
- Search mode
- Hybrid retrieval
- Semantic ranking
- Debugging ingestion failures
- Rebuilding only the index layer

Given your AWS 429 issue, this extra visibility is useful.

Bottom-line assessment

Yes, you should proceed with the Microsoft Foundry version now.

Your best first implementation path is:

```
Azure Blob Storage
→ Azure AI Search import/vectorize pipeline
→ text-embedding-3-small
→ Azure AI Search vector index
→ Foundry Chat Playground
→ Policy AI Assistant
```



Once that works, upgrade the architecture to:

```
Azure Blob Storage
→ Azure AI Search vector index
→ Foundry knowledge base / Foundry IQ
→ Foundry Agent
→ Policy AI Assistant with citations
```



This will let you keep making progress on your RAG Policy AI Assistant while AWS support investigates the Titan V2 / Bedrock 429 throttling issue.